Name: Yuvagiridharan U

Student ID No: 20089674

Module Instructor: Swati Dongre

Module Title: Advance programming Techniques

Word Count: 700 – 800

TABLE OF CONTENTS

Abstract **3**
Keywords **3**
I. Introduction 3

SECTION I — C# Implementation **3**
Task 1: Churros Food Truck Application **3**
1.1 Requirements and Design 3
1.2 OOP Concepts 3
1.3 Data Structure: Queue<Order> 3
1.4 pay_bill() Method 3
1.5 Unit Tests — pay_bill() 3

Task 2: Periodic Table Lookup System **4**
2.1 Design and Data Structure 4
2.2 Input Validation 4
2.3 Screenshot — Input Validator 4
2.4 Output of Periodic Table Execution 4

SECTION II — Python Implementation **4**
Task 3: EasyDrive TCP Client-Server **4**
3.1 System Architecture Overview 4
3.2 Client-Side Implementation 4
3.3 Server-Side Implementation 4
3.4 Input Validation — Client 4
3.5 Persistence and Registration Number 4

Task 4: Travel Books Web Scraper **5**
4.1 Design and Data Structure 5
4.2 HTML Parsing Strategy 5
4.3 Scraped Dataset 5
4.4 Web Scraper Output 6

Conclusion **6**
References

GIT-HUB Link: https://github.com/yuvagi-ridharan/20089674.git

Abstract This report presents the complete design, implementation, and testing of Continuous Assessment for Advanced Programming Techniques assessment comprises four tasks across two programming languages.

Section I covers
Task 1 implements a menu-driven food truck system using OOP principles with a generic Queue<Order> for FIFO order management and unit tests validating. Task 2 delivers a periodic table lookup tool using Dictionary<int, Element> for O(1) retrieval by atomic number. Together, both tasks demonstrate competency in modelling real-world workflows through structured OOP design and efficient data retrieval using advanced C# collection types.

Section II covers
Python Tasks 3 and 4: TCP client-server registration system for Easy Drive using socket programming, Tiny DB persistence, and UUID generation; and a web scraper extracting travel book data from books.toscrape.com

Keywords— C#, Python, Object-Oriented Programming, Queue, Dictionary, TCP/IP, Socket Programming, Tiny DB, BeautifulSoup4, Web Scraping, CSV, Unit Testing

I. INTRODUCTION

This report documents the complete implementation of Advanced Programming Techniques. The assessment comprises four tasks divided across C# (Section I) and Python (Section II).

SECTION I C# IMPLEMENTATION

Task 1: Churros Food ruck Application

1.1) Requirements and Design

Task1 requires a menu-driven console application simulating a food truck. Classes of **Churros** and **Order** are defined with OOP principles. A Queue<Order> (FIFO) is the primary data structure.

1.2) OOP Concepts

- **Encapsulation**: all fields private, access via C# properties with private setters.
- **Constructor**: parameterized; Order auto-increments Order No via static _nextOrderNo
- **Properties**: Name, Price (Churros); Order No, Bill, Quantity, Collected Order)
- **Access Modifiers**: private fields, internal Reset Counter () for test isolation.

*1.3) **Data Structure: Queue<Order>***

A Dictionary<int, Element> provides O(1) average-case lookup by atomic number. The Element class encapsulates AtomicNumber, Name, Symbol, Category, and Description.Dictionary was preferred overList (O(n) search) and plain

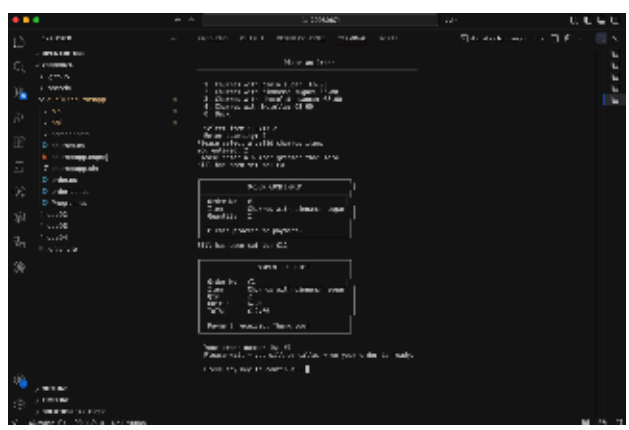


Fig 1.4. pay_bill() method (Order.cs)

1.5) Unit Tests — pay_bill()

Five tests in OrderTests.cs (no external framework). All pass.

```
(venv) yuvagiridharan@yuvas-MacBook-Air churrosapp % ls
bin                 churrosapp.sln          ordertest.cs
churros.cs          obj                     Program.cs
churrosapp.csproj   order.cs                screenshots
(venv) yuvagiridharan@yuvas-MacBook-Air churrosapp % dotnet run
/Users/yuvagiridharan/Documents/20089674/que01/churrosapp/order.cs(77,28): warning CS8602: Dereference of a po
ssibly null reference.
        Running Order Unit Tests
Please select a valid churros item.
You entered: 1
Please enter a number greater than zero.
Bill has been set to: €0
Bill has been set to: €6

        PAYMENT RECEIPT

  Order No : #1
  Item     : Churros with plain sugar
  Qty      : 1
  Unit €   : 6.00
  TOTAL    : €6.00

  Payment received. Thank you!

PASS — Test1: Single item bill is correct (6.00)
Please select a valid churros item.
You entered: 3
Please enter a number greater than zero.
Bill has been set to: €0
Bill has been set to: €24

        PAYMENT RECEIPT

  Order No : #1
  Item     : Churros with Nutella
  Qty      : 3
  Unit €   : 8.00
  TOTAL    : €24.00

  Payment received. Thank you!

PASS — Test2: Multiple quantity bill is correct (24.00)
Please select a valid churros item.
You entered: 1
Please enter a number greater than zero.
Bill has been set to: €0
Bill has been set to: €0
```

TABLE I. Task 1 Unit Test Results

Task 2: Periodic Table Lookup System

2.1) Design and Data Structure

A Dictionary<int, Element> provides O(1) average-case lookup by atomic number. The Element class encapsulates Atomic Number, Name, Symbol, Category, andDescription.Dictionary was preferred over-List (O(n) search) and plain array (wasted index 0).

*2.2) Input **Validation***

Two-layer validation: int.TryParse() rejects non-numeric input; a bounds check (atomicNumber < 1 || > 30) rejects out-of-range values. Both paths display informative messages and continue the loop without crashing.

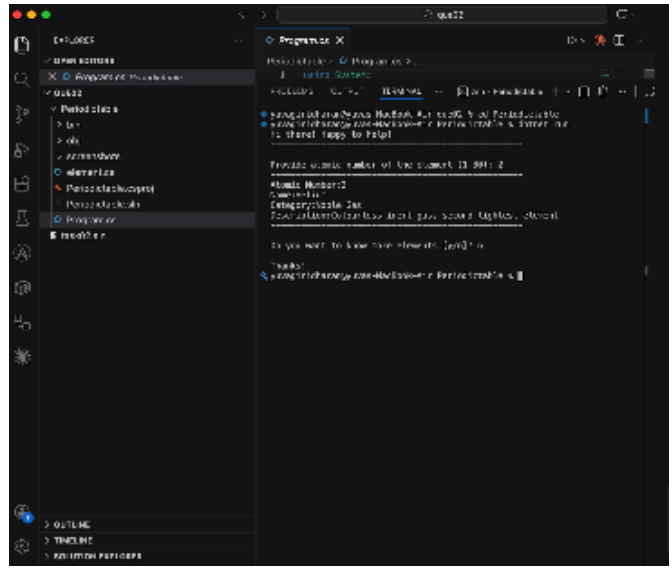
```
// ask to continue
Console.Write("\nDo you want to know more elements [y/n]? ");
again = Console.ReadLine()?.Trim().ToLower() ?? "n";

// validate y/n input
while (again != "y" && again != "n")
{
    Console.Write("Please enter y or n: ");
    again = Console.ReadLine()?.Trim().ToLower() ?? "n";
}
```

```
// validate input is a number
if (!int.TryParse(input, out int atomicNumber))
{
    Console.WriteLine("Invalid input. Please enter a number.");
    continue;
}

// validate number is in range
if (atomicNumber < 1 || atomicNumber > 30)
{
    Console.WriteLine("Please enter a number between 1 and 30.");
    continue;
}
```

2.3) Screenshot input validator



2.4) Output of periodic table Execution

SECTION II — PYTHON IMPLEMENTATION

Task 3: Easy Drive TCP Client-Server

3.1) **System Architecture Overview**

The system utilizes a **Client-Server Architecture** designed for reliable real-time data transmission. It is built upon the **Transmission Control Protocol (TCP)**to ensure that all registration data is received accurately and in the correct order

- **Network Layer:** The system uses the AF_INET address family (IPv4) and SOCK_STREAM socket type (TCP)
- **Communication Port:** Both entities communicate over port **65432** on the local host (127.0.0.1).

- **Data Interchange:** Information is exchanged using **JSON (JavaScript Object Notation)**, providing a lightweight and language-agnostic format for serializingcustomer objects. Component , Technology

3.2) **Client-Side Implementation(que03_client.py)**

TCP (AF_INET, SOCK_STREAM) client-server system using Python's socket library. The client (que03_client.py) collects and validates input using questionary, serialises to JSON, and transmits on port 65432. The server (que03_server.py) checks duplicate PPS, persists to Tiny DB, and returns a UUID registration number

3.3) **Server-Side Implementation(que03_Server.py)**)

The server acts as the central authority for processing, verifying persisting registration data.

3.4) Input Validation (Client)

Inline validators using questionnaire return True or an error string. validate_pps() enforces 7 digits + 1-2 letters; validate_name() uses regex for letters/spaces only.

3.5) Persistence and Registration Number

Tiny DB was chosen for its schema-free JSON storage and human-readable output. UUID4 generates a 12-character uppercase registration number. Three live records confirmed: yuva (D95D8681-611),Yuvagiri (9F603048-4BF), Sam (FBF18DB4-773)

```python
def save_customer(db, customer_data):
    # Generate unique registration number
    registration_number = str(uuid.uuid4()).upper()[:12]

    # Add extra info to customer data
    customer_data["registration_number"] = registration_number
    customer_data["registered_at"] = datetime.now().isoformat(timespec="seconds")

    # Get customers table and insert record
    customers_table = db.table("customers")
    customers_table.insert(customer_data)

    # Log the saved customer
    logger.info(
        "Saved customer %s with registration number %s",
        customer_data.get("name"),
        registration_number
    )

    return registration_number
```

Fig 3.6. save customer () with UUID (que03_server.py)

Task 4: Travel Books Web Scrape

4.1). Design and Data Structure

The scraper (que04.py) targets books. using Requests and BeautifulSoup4. Four functions implement single-responsibilitydesign: convert_raing(), scrape_books(), save_to_csv(), and read_from_csv().

4.2) HTML ***Parsing Strategy***

Title from anchor title attribute (avoids truncation); star rating from the second CSS class of <p class="star-rating">, converted to integer via convert rating (); price from <p class="price color"> text content.

4.3) Scraped ***Dataset***

Eleven travel books are extracted. Price range: £23.21 – £56.88 (avg £39.79). Rating distribution:

```
Saving data to books.csv...
Saved 11 books to books.csv

Reading data from books.csv...
=====================================================
Title                                       Rating   Price
=====================================================
It's Only the Himalayas                     2 stars  Â£45.17
Full Moon over Noahâs Ark: An Odyssey to M  4 stars  Â£49.43
See America: A Celebration of Our National P  3 stars  Â£48.87
Vagabonding: An Uncommon Guide to the Art of  2 stars  Â£36.94
Under the Tuscan Sun                        3 stars  Â£37.33
A Summer In Europe                          2 stars  Â£44.34
The Great Railway Bazaar                    1 stars  Â£30.54
A Year in Provence (Provence #1)            4 stars  Â£56.88
The Road to Little Dribbling: Adventures of  1 stars  Â£23.21
Neither Here nor There: Travels in Europe   3 stars  Â£38.95
1,000 Places to See Before You Die          5 stars  Â£26.08
=====================================================
Total books: 11
=====================================================
```

4.4) Web scrub output in execution

CONCLUSION

This report has documented the successful design, implementation, and verification of the four primary tasks comprising the CA1 assessment for Collectively, these tasks represent a of modern software engineering principles, transitioning from core object-oriented patterns to distributed network systems and persistent data management.

REFERENCES

- Troelsen, A. and Japikse, P. (2022). *Pro C# 10 with .NET 6*. Apress.

- Python Software Foundation. (2024). *Python Standard Library — socket, json, csv, uuid*.
- Mayer, J. (2024). *Tiny DB Documentation — Lightweight Document Database for Python*.
- Richardson, L. (2024). *Beautiful Soup Documentation — Python HTML Parsing Library*
- Books to Scrape. (2024). *Travel Books Category*.